Föreläsning 7

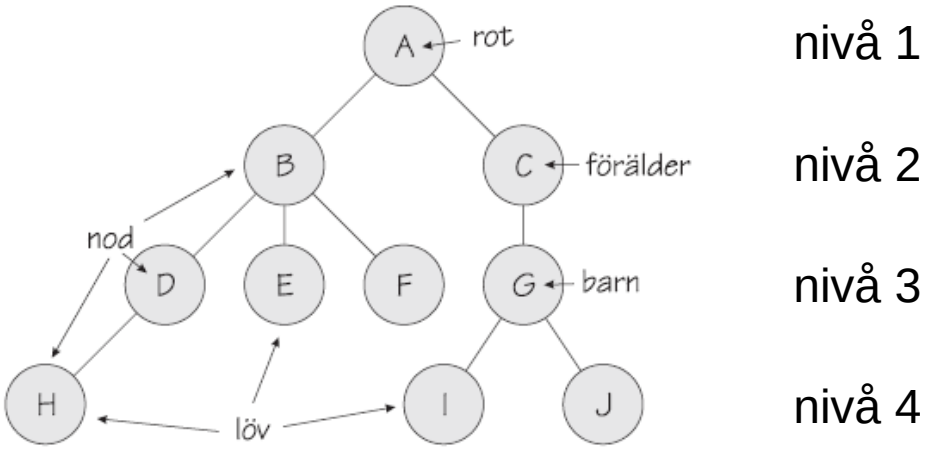
Träd och binära sökträd

Föreläsning 7

- Träd
- Binära träd
- Binära sökträd (BST)
- BST i JCF
- Implementering av BST

Det finns mycket matnyttigt i kapitel 6 som vi inte går igenom.

Terminologi - träd



Ett träd i datalogi består av en rotnod och ett ändligt antal underträd (subtrees).

Trädets höjd är antalet nivåer.

Ett träd är en graf där man kan ta sig mellan två noder på endast ett sätt.

N noder ger N-1 bågar.

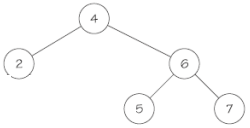
Vi kommer främst titta på binära träd.

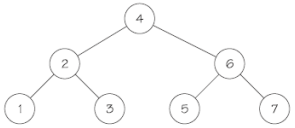
Binära träd

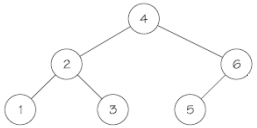
Ett binärt träd är ett träd där varje nod har maximalt 2 barn

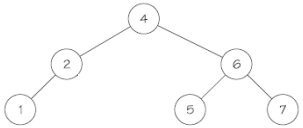
Definition:
Ett binärt träd är antingen tomt
eller
så har rotnoden 2 underträd (subtree)
som också är binära träd (vänster och höger underträd).

Full Perfekt Komplett Balanserat









För ett binärt träd gäller att:

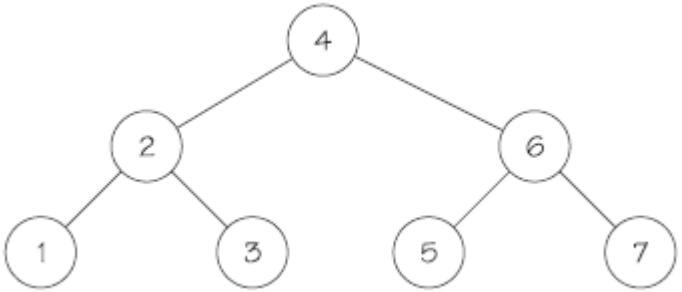
I ett fullt träd har varje nod antingen två barn eller inga barn.

I ett perfekt träd har alla interna noder två barn och alla löv befinner sig på samma nivå (alla nivåer är fulla).

I ett komplett träd är alla nivåer fulla utom möjligen den sista. I denna är noderna placerade så långt till vänster som möjligt.

I ett balanserat träd skiljer sig höjden för vänster och höger underträd till varje nod med maximalt 1.

Binärt sökträd (BST)



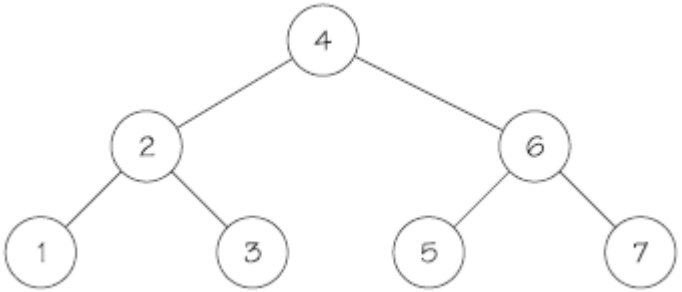
Ett binärt sökträd är ett binärt träd som är ordnat efter nodernas nycklar.

Definition:

- Ett binärt sökträd är antingen tomt eller så har rotnoden två subträd som också är binära sökträd.
- Alla nycklar i det vänstra subträdet är mindre än rotnodens nyckel och alla nycklar i det högra subträdet är större än rotnodens nyckel.

Definitionen ger att varje nyckel är unik. Vi kan inte ha dubbletter i ett binärt sökträd.

Poäng



Att söka i ett binärt sökträd är O(log(n)) om det är balanserat.

Att sätta in i ett binärt sökträd är O(1) när vi hittat platsen. Vi bara hänger på lövet.

Att söka i en sorterad array är också O(log(n)) men då är det O(n) att sätta in eller ta bort ett element.

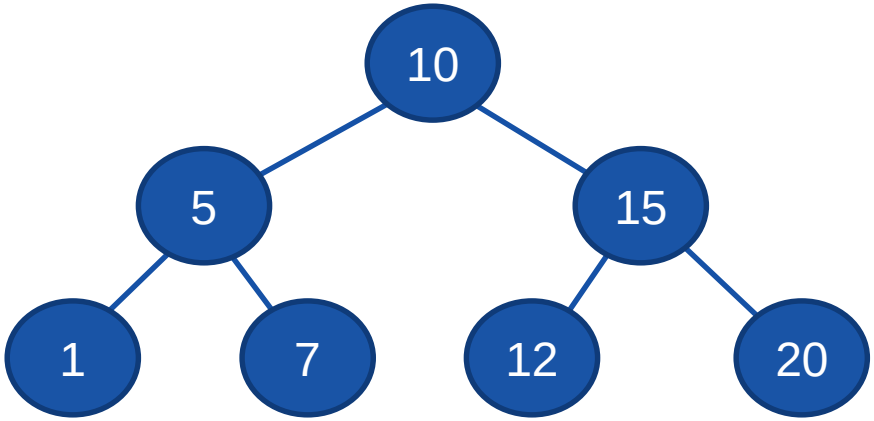
I en länkad lista är insättning och borttagning O(1) men där är binär sökning inte rimligt eftersom det är O(n) att hitta ett specifikt element.

Att bygga ett binärt sökträd

När vi sätter in nya element i trädet är det bara följa trädet ner.

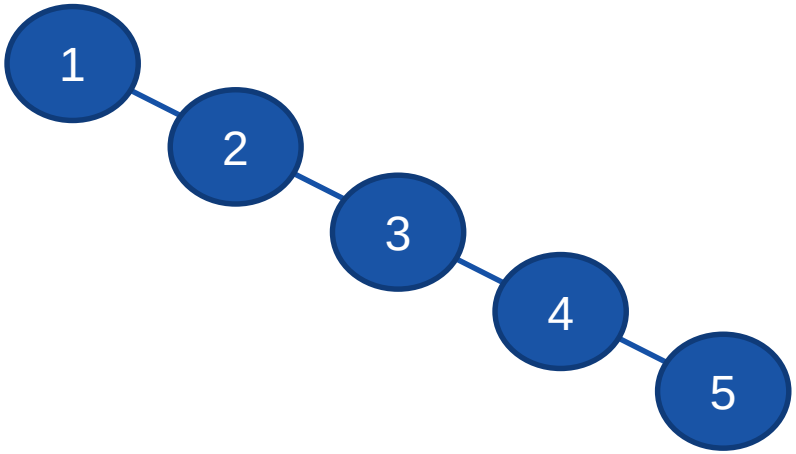
Bygg upp ett träd genom att sätta in elementen:

10, 5, 7, 15, 12, 1, 20 i denna ordning.



Självbalanserande binära sökträd

Prova nu att sätta in element med nycklarna 1,2,3,4,5 i angiven ordning

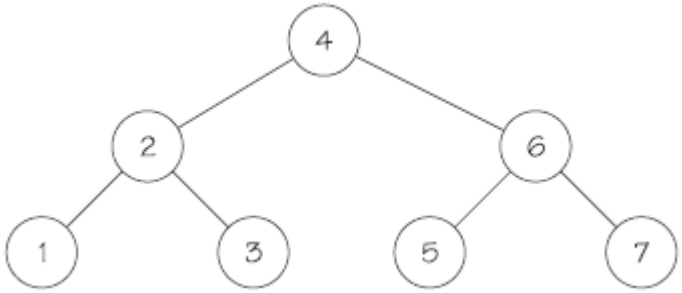


Resultatet blir ett binärt sökträd som är en länkad lista och där sökningen blir O(n) istället för O(log(n)).

För att undvika dylikt använder man självbalanserade träd.

Traversera

(besöka alla noder)



Tre traverseringsordningar (rekursiva):

- **Inorder**. Besök först trädets vänstra del, sedan noden själv och sist trädets högra del.

  (1,2,3,4,5,6,7)

- **Preorder**. Besök först noden själv, sedan trädets vänstra del och sist trädets högra del.

  (4,2,1,3,6,5,7)

- **Postorder**. Besök först trädets vänstra del, sedan trädets högra del och sist noden själv.

  (1,3,2,5,7,6,4)

Normalt använder vi inorder.

Vad ska vi använda ett BST till?

Vi kan använda ett BST för att implementera både en ordnad mängd (set) och en avbildning (map).

I en avbildning anger man till varje värde en nyckel som trädet sorteras efter. Man kan då söka efter ett värde mha en nyckel (tex söka efter telefonnummer mha namn). Nycklarna måste gå att ordna för att vi ska kunna använda ett BST.

Till en mängd adderar man värden och kan sedan få veta om ett visst värde finns i mängden. Värdena måste gå att ordna för att vi ska kunna använda ett BST.

Mer om mängder och avbildningar nästa gång…

Binärt sökträd i JCF

I java använder man tex ett binärt sökträd för att implementera en avbildning i klassen TreeMap. Denna implementerar ett Red-Black Tree vilket är ett självbalanserande binärt sökträd som garanterar O(log(n)). Exempel med vårt telefonregister:

```java
public static void main(String[] args) {
    Map<String,DirectoryEntry> phoneList = new TreeMap<>();
    while(true){
        String name = JOptionPane.showInputDialog("Namn eller enter:");
        if(name.equals(""))
            break;
        String number = JOptionPane.showInputDialog("Nummer:");
        DirectoryEntry d = new DirectoryEntry(name,number);
        phoneList.put(d.name,d);
    }
    while(true){
        String name = JOptionPane.showInputDialog("Vem vill du söka efter?");
        if(name.equals(""))
            break;
        DirectoryEntry found = phoneList.get(name);
        if(found!=null)
            System.out.println("Nummer: "+found.number);
        else
            System.out.println("Saknas");
    }
    System.out.println("Hej då");
}
```

```java
public class DirectoryEntry {
    public String name;
    public String number;

    public DirectoryEntry(String name, String number){
        this.name=name;
        this.number=number;
    }
}
```

Vårt BST

Vi ska följa boken och implementera en mängd (set) med hjälp av ett BST. Trots att det är ett set har det en find-metod snarare än en contains-metod.

Tanken är nog att vi kan använda den för att söka information genom att se till att compareTo för det vi stoppar in bara jämför delar av datat:

```
public class DirectoryEntry implements Comparable<DirectoryEntry> {
    public String name;
    public String number;

    public DirectoryEntry(String name, String number){
        this.name=name;
        this.number=number;
    }

    @Override
    public int compareTo(DirectoryEntry e){
        return name.compareTo(e.name);
    }
}
```

Implementering av BST

```
public class BinarySearchTree<E extends Comparable<E>> {

  private static class Node<E>{
    private E data;
    private Node<E> left,right;
    private Node(E d){
      data=d;
      left=right=null;
    }

    @Override
    public String toString(){
      return data.toString();
    }
  }

  private Node<E> root;

  public BinarySearchTree(){
    root=null;
  }
…
```

Garanterar att våra data är sorterbara

Traversera trädet inOrder

```
private void inOrder(Node<E> node, StringBuilder sb){
  if(node!=null){
    inOrder(node.left, sb);
    sb.append(": "+node.toString());
    inOrder(node.right, sb);
  }
}
public String toString(){
  StringBuilder sb = new StringBuilder();
  inOrder(root,sb);
  return sb.toString();
}
```

add

```
private boolean add(E data,Node<E> node){
  if(data.compareTo(node.data)==0)
    return false;
  else if(data.compareTo(node.data)<0)
    if(node.left==null){
      node.left = new Node<E>(data);
      return true;
    }else
      return add(data,node.left);
  else
    if(node.right==null){
      node.right = new Node<E>(data);
      return true;
    }else
      return add(data,node.right);
}
public boolean add(E data){
  if(root==null){
    root = new Node<E>(data);
    return true;
  }else
    return add(data,root);
}
```

find

```
private E find(E target, Node<E> node){
  if( node==null)
    return null;
  if(target.compareTo(node.data)==0)
    return node.data;
  if(target.compareTo(node.data)<0)
    return find(target,node.left);
  return find(target,node.right);
}

public E find(E target){
  return find(target, root);
}
```

Ta bort

Det finns tre fall att ta hänsyn till när vi tar bort en nod:

noden är ett löv

noden har bara ett subträd

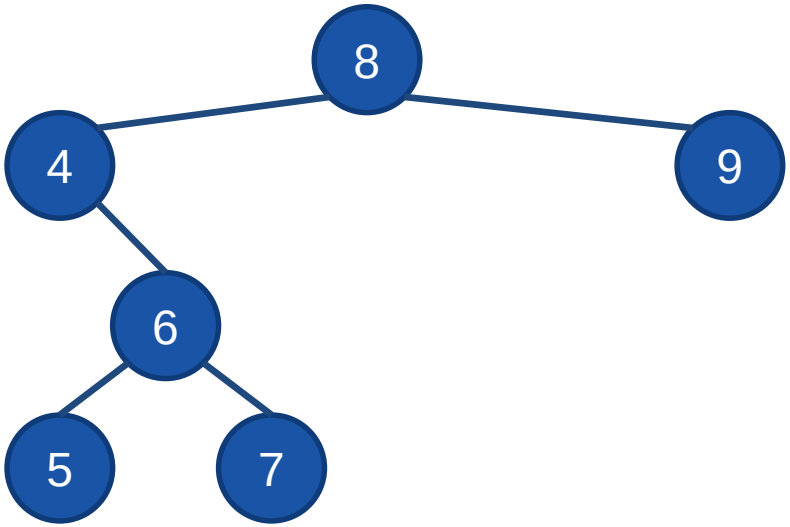
noden har både vänster och höger subträd

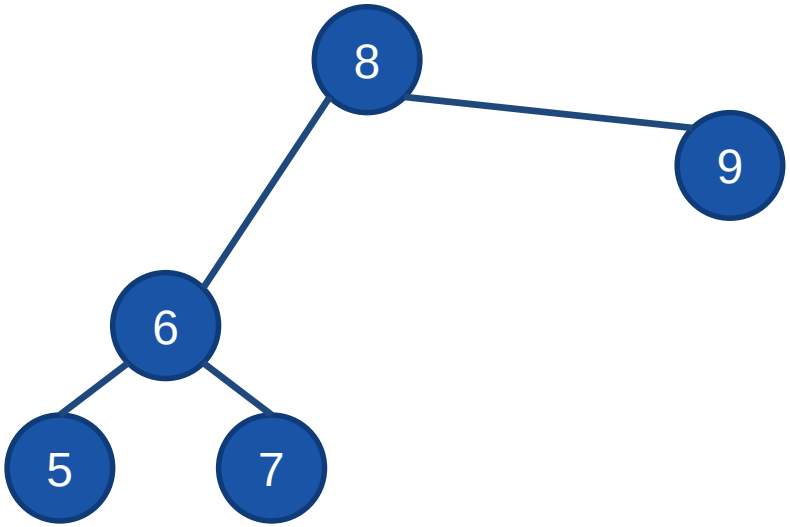
Fall 1 är trivialt: vi sätter helt enkelt förälderns relevanta subträd till NULL

Noden har bara ett subträd

Också relativt enkelt. Vi ersätter helt enkelt noden med dess barn.

Ex: Vi ska ta bort 4:





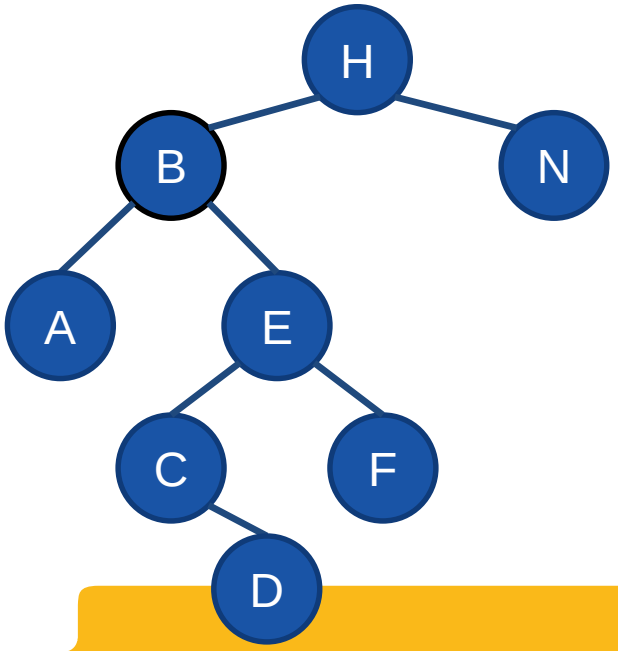
Noden har både vänster och höger subträd

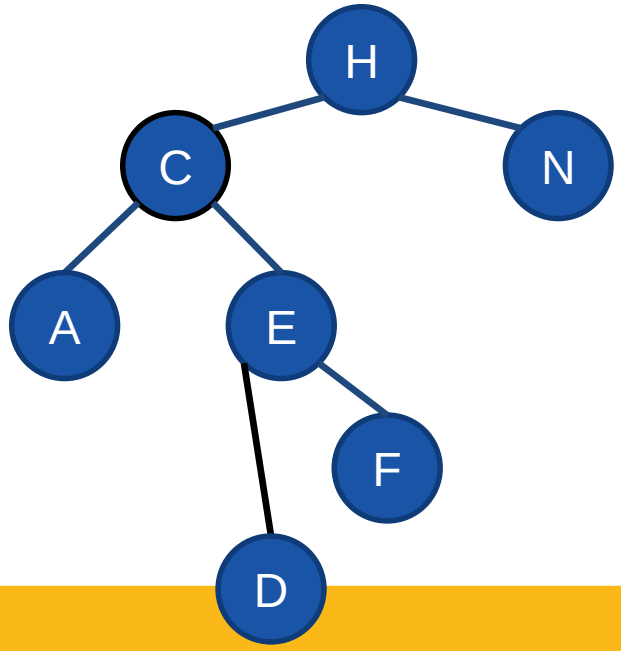
Vi måste nu lösa vad vi ska göra med de två barnen.

Lösningen är att vi ersätter noden med den minsta noden i det högra underträdet.

Denna kan inte ha något vänsterbarn (den är ju minst) och är därmed lätt att ta bort (fall 2)

Ex: vi vill ta bort nod B.





implementering

```
public E delete(E target){
  root = delete(target,root);
  return deletedData;
}
```

Eftersom vi ska ändra subträdet som har noden vi ska ta bort som rot är det enklast att rekursivt leta upp rätt nod och i varje steg returnera det nya trädet.

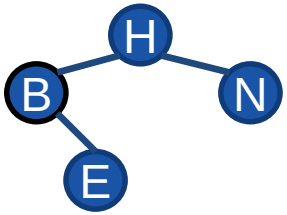
Dock behöver vår wrapper returnera datat från noden vi tar bort.

Detta kunde vi då gjort genom att anropa find först och spara detta men det skulle innebära att vi letar upp noden två gånger. Istället sparar vi datat i en privat medlemsvariabel i klassen:

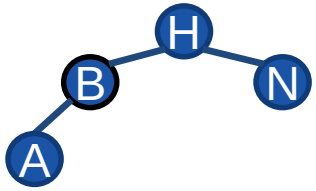
```
private E deletedData;
```

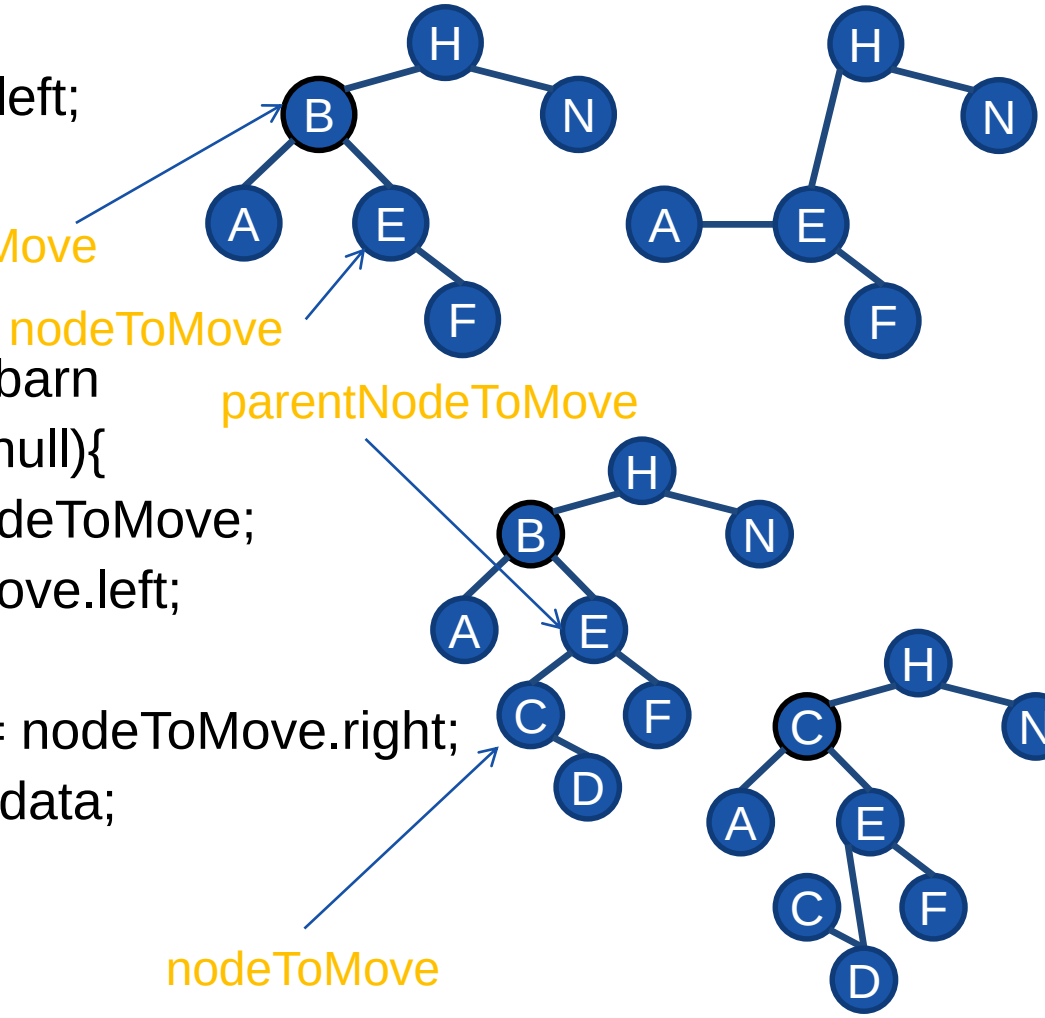
```
private Node<E> delete(E target,Node<E> node){

  if(node==null){//target finns ej i trädet
    deletedData = null;
    return null;
  }else{

    if(target.compareTo(node.data)<0){//target finns i vänstra trädet
      node.left=delete(target,node.left); //om det finns
      return node;
    }else if(target.compareTo(node.data)>0){//target i högra trädet
      node.right=delete(target,node.right);
      return node;
    }else{//target finns i node!
…
```

```
deletedData = node.data; //lagrar data att returnera
//nu ska vi bygga om trädet
if(node.left==null) //noden som ska bort saknar vänster träd
  return node.right;
```




```
else if(node.right==null)//noden som ska bort saknar högerträd
  return node.left;
```




```
else{//noden vi ska ta bort har två barn
…
```

```
Node<E> nodeToMove=node.right, parentNodeToMove=node;
 if(nodeToMove.left==null){//högra barnet har inget vänsterbarn

   nodeToMove.left=node.left;
   return nodeToMove;
 }

//högra barnet har vänsterbarn
 while(nodeToMove.left!=null){
   parentNodeToMove=nodeToMove;
   nodeToMove=nodeToMove.left;
 }
 parentNodeToMove.left = nodeToMove.right;
 node.data=nodeToMove.data;
 return node;
}}}}
```